

Complete NumPy & Pandas Function Reference

All 100 Stock Analysis Questions — Function Inventory & Cheat Sheet

1. NumPy Functions (by category)

1.1 Array Creation & Construction

Function	Purpose	Questions
<code>np.array()</code>	Build array from list	Setup
<code>np.concatenate()</code>	Join arrays (e.g., prepend NaN to returns)	Setup, Q14, Q99
<code>np.column_stack()</code>	Stack 1D arrays as columns (feature matrix)	Q99
<code>np.arange()</code>	Range of integers (x-axis for regression)	Q51
<code>np.full()</code>	Array filled with a constant (NaN container)	Q27, Q39, Q82, Q96
<code>np.zeros()</code>	Array of zeros (boolean mask)	Q93
<code>np.ones()</code>	Array of ones (adjustment factor)	Q75
<code>np.intersect1d()</code>	Common elements between two arrays (dates)	Q78
<code>np.searchsorted()</code>	Find insertion index (align dates across stocks)	Q78

1.2 Aggregation & Statistics

Function	Purpose	Questions
<code>np.max()</code> / <code>.max()</code>	Maximum value	Q1, Q9, Q11
<code>np.min()</code> / <code>.min()</code>	Minimum value	Q2, Q10, Q11
<code>np.mean()</code> / <code>.mean()</code>	Arithmetic mean	Q4, Q6, Q8, Q20, Q36, Q45
<code>np.median()</code>	Median	Q5
<code>np.std()</code> / <code>.std()</code>	Standard deviation	Q7, Q26, Q34
<code>np.var()</code>	Variance	Q83
<code>np.nanmax()</code>	Max ignoring NaN	Q16
<code>np.nanmin()</code>	Min ignoring NaN	Q17
<code>np.nanmean()</code>	Mean ignoring NaN	Q15, Q25, Q30, Q31, Q80, Q86–Q93, Q95, Q97, Q98
<code>np.nanstd()</code>	Std ignoring NaN	Q26, Q31, Q34, Q79, Q80, Q91
<code>np.nanmedian()</code>	Median ignoring NaN	(extended)
<code>np.nanpercentile()</code>	Percentile ignoring NaN	Q33, Q55
<code>np.percentile()</code>	Percentile	Q33 (alias)
<code>np.argmax()</code>	Index of max	Q3, Q25, Q37, Q43, Q77, Q79, Q80, Q86–Q89, Q91
<code>np.argmin()</code>	Index of min	Q29, Q38

Function	Purpose	Questions
<code>np.sum()</code>	Sum	Q18, Q19, Q35, Q42, Q58, Q62, Q68, Q69, Q71, Q72, Q76, Q100
<code>np.cumsum()</code>	Cumulative sum	(extended)
<code>np.cumprod()</code>	Cumulative product (equity curve)	(extended)
<code>np.maximum.accumulate()</code>	Running maximum (drawdown)	Q28, Q29
<code>np.minimum.accumulate()</code>	Running minimum	(extended)
<code>np.diff()</code>	Consecutive differences	Setup, Q14
<code>np.unique()</code>	Unique values	Setup (months/years)

1.3 Correlation & Covariance

Function	Purpose	Questions
<code>np.corrcoef()</code>	Pearson correlation matrix	Q41, Q78, Q82, Q94, Q95, Q96
<code>np.cov()</code>	Covariance matrix	Q83

1.4 Boolean & Logical Operations

Function	Purpose	Questions
<code>np.sum(mask)</code>	Count True in boolean mask	Q18, Q19, Q35, Q58, Q62
<code>np.mean(mask)</code>	Fraction True	Q20, Q65
<code>np.where()</code>	Indices where condition is True	Q46, Q47, Q48, Q60, Q61, Q85
<code>np.isnan()</code>	NaN detection	Q41, Q50, Q51, Q67, Q78, Q95
<code>np.isin()</code>	Membership test (months in May–Oct)	Q92
<code>np.logical_and / &</code>	Element-wise AND	Q46–Q48, Q60, Q61, Q64
<code>np.logical_or / </code>	Element-wise OR	Q64, Q65
<code>np.logical_not / ~</code>	Element-wise NOT	Q50, Q55, Q78, Q92
<code>np.nan_to_num()</code>	Replace NaN with a number	Q63

1.5 Window & Rolling Operations (Manual)

Pattern	Purpose	Questions
<code>for i in range(w-1, len(arr)): arr[i-w+1:i+1].mean()</code>	Rolling mean	Q39
<code>for i in range(w-1, len(arr)): arr[i-w+1:i+1].std(ddof=1)</code>	Rolling std	Q27
<code>for i in range(w-1, len(arr)): np.corrcoef(...)[0,1]</code>	Rolling correlation	Q82
<code>np.polyfit(x, y, 1)[0]</code>	Linear regression slope	Q51

1.6 Indexing & Slicing

Pattern	Purpose	Questions
<code>arr[mask]</code>	Boolean indexing	Everywhere
<code>arr[a:b]</code>	Slice	Q9, Q10, Q73, Q98
<code>arr[1:], arr[:-1]</code>	Shift-by-one slicing	Q14, Q46, Q47, Q48, Q60, Q61
<code>arr[-1], arr[-252:]</code>	Tail access	Q9, Q10, Q49, Q52, Q53, Q54, Q59, Q66, Q73
<code>arr[np.argmax(...)]</code>	Value by index	Q3, Q37
<code>np.where(...)[0] + 1</code>	Offset index for cross detection	Q46, Q47, Q48, Q60, Q61

1.7 Date/Time (used with Python datetime)

Pattern	Purpose	Questions
<code>np.array([datetime(...) for ...])</code>	Build datetime array	Setup
<code>d.month, d.year, d.weekday()</code>	Extract parts	Q22–Q25, Q43, Q86–Q92
<code>np.array([d.month for d in dates])</code>	Month array	Q22, Q25, Q43, Q87, Q91
<code>np.array([d.weekday() for d in dates])</code>	Weekday array	Q86, Q90

1.8 Math Utilities

Function	Purpose	Questions
<code>np.sqrt()</code>	Square root (annualization)	Q26, Q30, Q31
<code>np.polyfit()</code>	Polynomial fit (slope)	Q51
<code>np.diff()</code>	First difference	Setup, Q14
<code>np.abs()</code>	Absolute value	Q85
<code>np.floor_divide / //</code>	Integer division (quarters)	Q88

2. Pandas Functions (by category)

2.1 I/O & DataFrame Construction

Function	Purpose	Questions
<code>pd.read_csv()</code>	Load CSV	Setup
<code>pd.DataFrame()</code>	Build DataFrame	Setup
<code>df.sort_values()</code>	Sort by column	Setup
<code>df.reset_index()</code>	Reset index	Setup
<code>df.dropna()</code>	Drop missing rows	(cleaning)
<code>df.fillna()</code>	Fill missing values	(cleaning)

2.2 Inspection

Function	Purpose	Questions
<code>df.head()</code>	First N rows	Q1–Q5
<code>df.tail()</code>	Last N rows	Q9, Q10, Q73
<code>df.shape</code>	Dimensions	Setup
<code>df.columns</code>	Column names	Setup
<code>df.dtypes</code>	Data types	Setup
<code>df.describe()</code>	Summary stats	Setup
<code>df.info()</code>	Schema info	Setup

2.3 Column Access & Selection

Function	Purpose	Questions
<code>df['Close']</code>	Column access	Everywhere
<code>df[['A', 'B']]</code>	Multi-column	Q53
<code>df.loc[...]</code>	Label-based indexing	Q3, Q24, Q27, Q98
<code>df.iloc[...]</code>	Integer-based indexing	Q13, Q21, Q49, Q52, Q53, Q54, Q59, Q66, Q73
<code>df.at[...]</code>	Scalar access	(extended)

2.4 Aggregation

Function	Purpose	Questions
<code>.max()</code>	Max	Q1, Q9, Q11
<code>.min()</code>	Min	Q2, Q10, Q11
<code>.mean()</code>	Mean	Q4, Q6, Q8, Q15, Q20, Q36, Q45
<code>.median()</code>	Median	Q5
<code>.std()</code>	Std dev	Q7, Q26, Q34

Function	Purpose	Questions
.sum()	Sum	Q18, Q19, Q35, Q42, Q58, Q62, Q69, Q71, Q72
.count()	Count non-NaN	Q71
.quantile()	Quantile	Q33, Q55
.idxmax()	Index of max	Q3, Q25, Q37, Q43, Q77, Q79, Q86, Q87, Q89, Q91
.idxmin()	Index of min	Q29, Q38
.nunique()	Number of unique values	(exploration)
.value_counts()	Value frequency	(exploration)

2.5 Transformations

Function	Purpose	Questions
.pct_change()	Percent change	Q14–Q20, Q24, Q25, Q41, Q44, Q86–Q88, Q96, Q97, Q100
.pct_change(5)	5-period percent change	Q96, Q100
.diff()	Difference	Q44
.shift(k)	Lag/shift	Q46, Q47, Q60, Q61, Q94, Q95, Q96, Q99
.shift(-k)	Forward shift	Q94, Q95, Q96, Q100
.cummax()	Running max	Q28, Q29
.cumsum()	Running sum	(extended)
.abs()	Absolute value	Q32
.apply()	Apply function	(feature engineering)
.astype()	Cast type	(cleaning)
.rename()	Rename columns	(cleaning)

2.6 Rolling & Window Operations

Function	Purpose	Questions
.rolling(30).std()	Rolling std	Q27
.rolling(20).mean()	Rolling mean	Q39
.rolling(30).corr(other)	Rolling correlation	Q82

2.7 Grouping & Aggregation

Function	Purpose	Questions
.groupby()	Group by key	Q25, Q43, Q44, Q86, Q87, Q88, Q91, Q92
.groupby(...).mean()	Group mean	Q25, Q86, Q87, Q88, Q91, Q92
.groupby(...).sum()	Group sum	Q43, Q71

Function	Purpose	Questions
<code>.groupby(...).std()</code>	Group std	Q91
<code>.groupby(...).max()</code>	Group max	(extended)
<code>.groupby(...).idxmax()</code>	Group argmax	Q25, Q43, Q86, Q87, Q91
<code>.groupby(...).diff()</code>	Group difference	Q44

2.8 Time Series (Resampling)

Function	Purpose	Questions
<code>.resample('M')</code>	Monthly resample	Q22
<code>.resample('Y')</code>	Yearly resample	Q23, Q89
<code>.resample('M').last()</code>	Monthly last value	Q22
<code>.resample('Y').last()</code>	Yearly last value	Q23, Q89
<code>.resample('M').last().pct_change()</code>	Monthly return	Q22
<code>.resample('Y').last().pct_change()</code>	Yearly return	Q23, Q89
<code>.dt.month</code>	Month component	Q25, Q43, Q87, Q91, Q92
<code>.dt.year</code>	Year component	Q23, Q76, Q89
<code>.dt.quarter</code>	Quarter component	Q88
<code>.dt.day_name()</code>	Weekday name	Q86
<code>.dt.dayofweek</code>	Weekday integer	Q90
<code>.dt.is_month_end</code>	Month-end flag	Q93

2.9 Correlation & Covariance

Function	Purpose	Questions
<code>.corr()</code>	Pearson correlation	Q41, Q78, Q94, Q95, Q96
<code>.cov()</code>	Covariance	Q83

2.10 Merging & Joining

Function	Purpose	Questions
<code>pd.merge()</code>	Join DataFrames	Q78, Q82
<code>.join()</code>	Join on index	Q78
<code>pd.concat()</code>	Stack DataFrames	Q99

2.11 Boolean Filtering

Pattern	Purpose	Questions
<code>df[df['Close'] > 70]</code>	Row filter	Q56, Q57
<code>df[(cond1) & (cond2)]</code>	Multiple conditions	Q46, Q47, Q48, Q60, Q61, Q64
<code>df[(cond1) (cond2)]</code>	OR conditions	Q64, Q65
<code>df.loc[mask, 'col']</code>	Filter with column select	Q24
<code>(df['Close'] > 70; df['MA_50']).sum()</code>	Count mask	Q50, Q58, Q62, Q71

2.12 Window / Cross Detection

Pattern	Purpose	Questions
<code>df[(df['A']>df['B']) & (df['A'].shift()<=df['B'].shift())]</code>	Cross above	Q46, Q47, Q60
<code>df[(df['A']<df['B']) & (df['A'].shift()>=df['B'].shift())]</code>	Cross below	Q48, Q61

2.13 Indexing for Time Series

Pattern	Purpose	Questions
<code>df.loc['2024:', 'Close']</code>	Slice by date	Q24
<code>df.loc[df['Close'].idxmax(), 'Date']</code>	Value by argmax	Q3
<code>df.loc[df['Volume'].idxmax(), 'Date']</code>	Value by argmax	Q37
<code>df.loc[df['Volume'].idxmin(), 'Date']</code>	Value by argmin	Q38

2.14 Mathematical Operations on Columns

Operation	Purpose	Questions
<code>df['High'] - df['Low']</code>	Column subtraction	Q6, Q7, Q55
<code>df['Close'] - df['Open']</code>	Gap	Q8
<code>df['High'] + df['Low']</code>	Sum	Q12
<code>(df['High'] + df['Low']) / 2</code>	Mid-price	Q12
<code>df['Close'] * df['Volume']</code>	Weighted price	Q45
<code>df['Close'] / df['Close'].cummax()</code>	Drawdown ratio	Q28, Q29
<code>df['Close'] / df['MA_20'] - 1</code>	Distance from MA	Q52
<code>df['Upper_Band'] - df['Lower_Band']</code>	Bandwidth	Q55, Q66
<code>df['Dividends'].tail(4).sum() / df['Close'].iloc[-1]</code>	Dividend yield	Q73

3. Cross-Reference — Function → Questions

NumPy

Function	Questions
<code>np.array</code>	Setup
<code>np.argmax</code>	3, 25, 37, 43, 77, 79, 80, 86–89, 91
<code>np.argmin</code>	29, 38
<code>np.corrcoef</code>	41, 78, 82, 94, 95, 96
<code>np.cov</code>	83
<code>np.concatenate</code>	Setup, 14, 99
<code>np.column_stack</code>	99
<code>np.cumsum</code>	extended
<code>np.cumprod</code>	extended
<code>np.diff</code>	Setup, 14
<code>np.full</code>	27, 39, 82, 96
<code>np.intersect1d</code>	78
<code>np.isin</code>	92
<code>np.isnan</code>	41, 50, 51, 67, 78, 95
<code>np.logical_and</code> / <code>&</code>	46–48, 60, 61, 64
<code>np.logical_or</code> / <code> </code>	64, 65
<code>np.logical_not</code> / <code>~</code>	50, 55, 78, 92
<code>np.max</code>	1, 9, 11
<code>np.maximum.accumulate</code>	28, 29
<code>np.mean</code>	4, 6, 8, 20, 36, 45
<code>np.median</code>	5
<code>np.min</code>	2, 10, 11
<code>np.minimum.accumulate</code>	extended
<code>np.nan_to_num</code>	63
<code>np.nanmax</code>	16
<code>np.nanmean</code>	15, 25, 30, 31, 80, 86–93, 95, 97, 98
<code>np.nanmedian</code>	extended
<code>np.nanmin</code>	17
<code>np.nanpercentile</code>	33, 55
<code>np.nanstd</code>	26, 31, 34, 79, 80, 91
<code>np.percentile</code>	33
<code>np.polyfit</code>	51
<code>np.searchsorted</code>	78

Function	Questions
np.sqrt	26, 30, 31
np.std	7, 26, 34
np.sum	18, 19, 35, 42, 58, 62, 68, 69, 71, 72, 76, 100
np.var	83
np.where	46–48, 60, 61, 85
np.zeros	93
np.ones	75
np.unique	Setup
np.arange	51

Pandas

Method	Questions
<code>.abs()</code>	32
<code>.apply()</code>	feature engineering
<code>.astype()</code>	cleaning
<code>.at</code>	extended
<code>.columns</code>	Setup
<code>.corr()</code>	41, 78, 94, 95, 96
<code>.count()</code>	71
<code>.cov()</code>	83
<code>.cummax()</code>	28, 29
<code>.cumsum()</code>	extended
<code>.describe()</code>	Setup
<code>.diff()</code>	44
<code>.dropna()</code>	cleaning
<code>.dtypes</code>	Setup
<code>.dt.day_name()</code>	86
<code>.dt.dayofweek</code>	90
<code>.dt.is_month_end</code>	93
<code>.dt.month</code>	25, 43, 87, 91, 92
<code>.dt.quarter</code>	88
<code>.dt.year</code>	23, 76, 89
<code>.fillna()</code>	cleaning
<code>.groupby()</code>	25, 43, 44, 86–88, 91, 92
<code>.head()</code>	Q1–Q5
<code>.idxmax()</code>	3, 25, 37, 43, 77, 79, 86, 87, 89, 91
<code>.idxmin()</code>	29, 38
<code>.iloc</code>	13, 21, 49, 52–54, 59, 66, 73
<code>.info()</code>	Setup
<code>.join()</code>	78
<code>.loc</code>	3, 24, 27, 98
<code>.max()</code>	1, 9, 11
<code>.mean()</code>	4, 6, 8, 15, 20, 36, 45
<code>.median()</code>	5
<code>.min()</code>	2, 10, 11
<code>.nunique()</code>	exploration
<code>.pct_change()</code>	14–20, 24, 25, 41, 44, 86–88, 96, 97, 100

Method	Questions
.quantile()	33, 55
.rename()	cleaning
.resample('M')	22
.resample('Y')	23, 89
.reset_index()	Setup
.rolling(20).mean()	39
.rolling(30).corr()	82
.rolling(30).std()	27
.shape	Setup
.shift()	46, 47, 60, 61, 94, 95, 96, 99
.shift(-k)	94, 95, 96, 100
.sort_values()	Setup
.std()	7, 26, 34
.sum()	18, 19, 35, 42, 58, 62, 69, 71, 72
.tail()	9, 10, 73
.value_counts()	exploration
.var()	83
pd.concat()	99
pd.DataFrame()	Setup
pd.merge()	78, 82
pd.read_csv()	Setup

4. Summary Counts

NumPy

Category	Unique Functions
Creation	10
Aggregation	22
Correlation	2
Boolean	10
Window (manual)	3
Indexing	7
Math	6
Total unique	~40

Pandas

Category	Unique Methods
I/O	3
Inspection	6
Selection	5
Aggregation	12
Transformations	10
Rolling	3
Grouping	6
Time Series	8
Correlation	2
Merging	3
Boolean	5
Window/Cross	2
Total unique	~55

5. The 20 Most-Used Functions Across All 100 Questions

Rank	Function	Count of Q's
1	np.mean / .mean()	~20
2	np.nanmean	~15
3	.pct_change()	15
4	np.std / .std()	~10
5	.groupby()	8
6	np.argmax / .idxmax()	10
7	.shift()	8
8	np.corrcoef / .corr()	6
9	np.sum() / .sum()	12
10	.iloc	7
11	np.where()	6
12	.rolling()	3
13	.resample()	3
14	.dt.month	5
15	np.max / .max()	3
16	np.min / .min()	3
17	np.nanstd	6
18	np.isnan	6
19	.loc	4
20	.diff()	2

6. Functions You Should Memorize

If you're learning this stack for finance, these 15 functions cover 80% of the handbook:

```
# NumPy
np.mean np.std np.median          # basic stats
np.nanmean np.nanstd              # NaN aware
np.diff np.concatenate            # returns
np.corrcoef np.cov                # relationships
np.argmax np.argmin               # locate extremes
np.maximum.accumulate             # running max (drawdown)
np.where                           # conditional index
np.polyfit                        # slope

# Pandas
not change()                      # returns
rolling(w).mean() / std()         # windows
shift(k)                           # lag
groupby().agg()                   # grouping
resample('M') last()              # time series
dt.month / dt.year / dt.dayofweek # calendar
corr()                             # correlation
idxmax() / idxmin()               # locate extremes
loc / iloc                         # selection
.cummax()                          # running max
```

7. Cheat Sheet — NumPy vs Pandas Equivalents

Task	NumPy	Pandas
Mean	<code>arr.mean()</code>	<code>s.mean()</code>
Std	<code>arr.std(ddof=1)</code>	<code>s.std()</code>
Median	<code>np.median(arr)</code>	<code>s.median()</code>
Percentile	<code>np.percentile(arr, 5)</code>	<code>s.quantile(0.05)</code>
Argmax	<code>np.argmax(arr)</code>	<code>s.idxmax()</code>
Argmin	<code>np.argmin(arr)</code>	<code>s.idxmin()</code>
Diff	<code>np.diff(arr)</code>	<code>s.diff()</code>
Percent change	<code>np.diff(a)/a[:-1]</code>	<code>s.pct_change()</code>
Lag	manual shift	<code>s.shift(k)</code>
Rolling mean	manual loop	<code>s.rolling(w).mean()</code>
Rolling std	manual loop	<code>s.rolling(w).std()</code>
Cumulative max	<code>np.maximum.accumulate(a)</code>	<code>s.cummax()</code>
Correlation	<code>np.corrcoef(a,b)[0,1]</code>	<code>a.corr(b)</code>
Group by	boolean masks	<code>df.groupby(...)</code>
Resample	manual	<code>df.resample('M').last()</code>
NaN mean	<code>np.nanmean(a)</code>	<code>s.mean()</code> (auto)
Boolean count	<code>np.sum(a > 0)</code>	<code>(s > 0).sum()</code>
Filter rows	<code>a[a > 0]</code>	<code>s[s > 0]</code>

Final Note: ~40 NumPy functions + ~55 Pandas methods = the complete toolkit for all 100 questions. NumPy shines for math, boolean masks, and tight loops. Pandas shines for labeling, grouping, time series, and missing data. Many questions can be answered in either — pick the one that reads more naturally.